Rollin' Down the Biggest Hill

CS 351 Analysis of Algorithms, Project Checkpoint

The WHEEL Ones: Conor Vahey, Felicity Chin, Oscar Yaun

Repository: `https://github.com/qcnr/Heely-Analysis` Analysis draft: separate PDF (`report/analysis_draft.pdf`)

1 (a) The question, and what changed since the proposal

Question. Given a map with elevation, find the best route between two points for someone on Heelys, and compare how much work Dijkstra's algorithm and ALT (A*, Landmarks, Triangle inequality) do to find it. The map is a directed graph, where rolling downhill costs 0, and a flat or uphill stretch costs 1 (one "push"). The best route is the one with the fewest pushes. Both algorithms must return a route with that cost, and we compare their running times.

Changes, and why.

1. *Objective: from "highest downhill ratio" to "fewest pushes."* A ratio is not a sum of edge costs, so it is not a shortest-path problem, and neither algorithm can find it directly. "Fewest pushes" keeps the same idea (roll, don't push) and is a sum of non-negative edge costs, which is what both algorithms need.
2. *Weights flipped.* The proposal gave downhill a weight of 1 and everything else 0. We use downhill $= 0$ and flat/uphill $= 1$, so that the shortest path is the route we actually want.
3. *Schedule.* The proposal had both algorithms started by Nov 4. Instead, we focused on getting everything for Dijkstra worked out, because ALT will use a lot of the same code that we made for Dijkstra. Section (d) has the updated plan.

2 (b) What is implemented, and how we know it is correct

All code is Python. Implemented:

- `heely/graph.py`: generates an $n \times n$ grid with random hills, with edge costs as above. Seeded, so each grid can be rebuilt exactly.
- `heely/dijkstra.py`: our own Dijkstra with a binary heap, stopping when the goal is reached. Returns the route, its cost, and how many nodes it settled.
- `heely/experiment.py` and `heely/plot.py`: timing code and the plot in Section (c).

Evidence of correctness (`python -m pytest -q`), we had AI make 10 tests, as well as hand checking smaller grids:

- *Hand-checked map.* A 3×3 hill where the right answers can be worked out by hand (peak to any corner is 0 pushes, corner to peak is 2).
- *Brute force on small grids.* On five grids from 2×2 to 4×4, for all 690 start/goal pairs, Dijkstra's cost matches the cheapest of every possible path.
- *Larger grids.* Brute force is too slow past 4×4, so on grids up to 60×60 we check Dijkstra's distances directly: the start has distance 0, no edge offers a shortcut ($d[v] \leq d[u] + w(u, v)$ for every edge), and every node's distance is reached by a real edge from its parent. Distances that pass these checks are the true shortest distances (a standard check; see CLRS Chapter 22), so no second algorithm is needed. All 9 full runs pass.

3 (c) Preliminary results

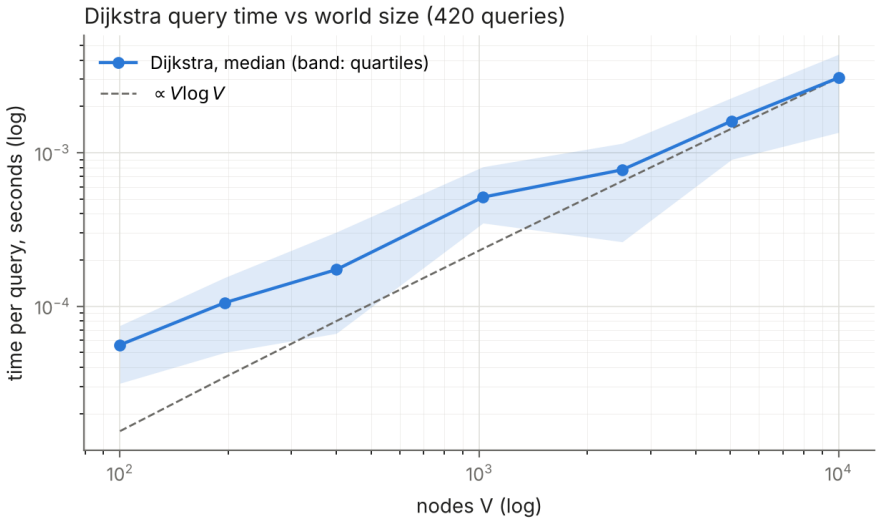


Figure 1: How long Dijkstra takes to find a route as the grid gets bigger. We tested seven grid sizes, from 10×10 (100 nodes) to 100×100 (10,000 nodes). For each size we made 3 random grids and ran 20 random start-to-goal searches on each. Both axes use a log scale, and the solid line is the median time at each size, while the shaded band shows the middle half of the times. The dashed line shows how $V \log V$ grows, for comparison with the expected running time.

- Query time grows in line with the $V \log V$ reference, as the analysis predicts: from a mean of 0.05 ms at 100 nodes to 3.0 ms at 10,000 nodes.
- On average Dijkstra settles about half of all nodes before reaching the goal (44% to 54% across the sizes), because it spreads out evenly in every direction instead of heading toward the goal. When we run with ALT, the number of nodes we search should reduce.
- The spread is wide because the cost of a query depends a lot on where the start and goal are relative to the hills.

4 (d) What remains, and the schedule

- **By Nov 16.** Implement ALT (choose landmarks, precompute distances to and from them, run A*), test that it gives the same cost as Dijkstra on every query, and add it to the timing code so both run on the same grids and queries.
- **By Nov 20.** Run the full comparison and make the final plots. Write the ALT correctness and running-time analysis. Try the real-world CoMaps/GPX data; if it is not working by Nov 20, it becomes a future-work note, as planned in the proposal.
- **By Nov 30.** Write the final report and presentation.

5 (e) Contributions

As a group, we wrote drafts of the project checkpoint, analysis draft, and all of our code. Then, Conor went over the project checkpoint and refined it, Felicity went over the analysis draft and refined it, while Oscar reviewed our code, made comments, and debugged (and rebugged) our code, as well as seperating files, so Conor could neatly upload everything to GitHub.